Universidade Católica de Brasília

Ciência da Computação

Arthur Freitas Araújo
Arthur Martins Fonseca Dias
Caio Seabra de Queiroz
Marina Souza Ramos

Arquiteturas RISC vs. CISC: características, aplicações e estudo de caso

Relatório técnico-acadêmico apresentado à Universidade Católica de Brasília, como parte dos requisitos avaliativos da unidade curricular de Arquitetura de Computadores.
Professor: Weldes Lima Oliveira.

Brasília – DF
2026

Sumário

1 Introdução . . . 3
1.1 Contextualização . . . 3
1.2 Importância para o desempenho . . . 3
1.3 Objetivos . . . 3
1.3.1 Objetivo geral . . . 3
1.3.2 Objetivos específicos . . . 3
1.4 Organização do trabalho . . . 4
2 Fundamentação teórica . . . 5
2.1 Arquitetura de conjunto de instruções (ISA) . . . 5
2.2 Filosofia RISC . . . 5
2.3 Filosofia CISC . . . 5
2.3.1 Arquitetura memória-memória . . . 5
2.3.2 Arquitetura registrador-memória . . . 6
2.3.3 Características da arquitetura CISC . . . 6
2.4 Densidade de código e complexidade de decodificação . . . 7
2.5 Consumo de energia e contagem de transistores . . . 7
2.6 Pipeline . . . 8
2.7 Convergência moderna . . . 8
3 Mapeamento e diagrama arquitetural . . . 9
3.1 Fluxo de execução de instruções . . . 9
3.2 Fluxo de instrução em RISC e em CISC . . . 9
3.3 Registradores das arquiteturas da PoC . . . 9
3.4 Diagrama de pipeline comparativo . . . 10
4 Metodologia e desenvolvimento . . . 11
4.1 Ambiente e ferramentas . . . 11
4.2 Arquiteturas selecionadas . . . 11
4.3 Código-fonte em C . . . 11
4.4 Assembly gerado . . . 11
4.4.1 x86-64 . . . 11
4.4.2 ARM64 . . . 13
4.4.3 Comentários sobre as listagens . . . 14
4.5 Passo a passo da PoC . . . 14
4.6 Métricas coletadas . . . 15
4.7 Limitação do método . . . 16
5 Discussão dos resultados . . . 17
5.1 Métricas obtidas . . . 17

5.2 Análise da comparação 17
5.3 Do código em C ao hardware 17
5.4 Limitações do método 17
5.5 Limitações físicas 18
5.6 Estudo de caso: servidores e dispositivos móveis 18
5.7 Impacto no desenvolvimento de software de alto desempenho 18
6 Conclusão 20
6.1 Síntese do aprendizado 20
6.2 Lições aprendidas 20
6.3 Implicações práticas 20

Referências 21

1 Introdução

1.1 Contextualização

O conjunto de instruções (ISA, *Instruction Set Architecture*) é a interface entre o software e o hardware. Ele define as instruções que o processador executa, os tipos de operandos, os modos de endereçamento e os registradores acessíveis ao programador (STALLINGS, 2017). Duas filosofias de projeto de ISA se consolidaram: RISC (*Reduced Instruction Set Computer*) e CISC (*Complex Instruction Set Computer*).

A filosofia RISC prioriza instruções simples e regulares, com o acesso à memória separado das operações aritméticas (STALLINGS, 2017). A filosofia CISC admite instruções mais complexas, de formato variável, que podem operar diretamente sobre a memória (SILVA; BORGES, 2024). Hoje a fronteira entre as duas é menos nítida: os processadores da linha x86 aceitam o conjunto de instruções CISC, mas o traduzem, em tempo de execução, para operações internas mais simples (SILVA; BORGES, 2024).

1.2 Importância para o desempenho

O tempo de execução de um programa depende do número de instruções, dos ciclos gastos por instrução e da duração do ciclo de relógio (Equação 2.1). As duas filosofias atuam sobre esses fatores de modos diferentes. Instruções complexas tendem a reduzir o número de instruções do programa, enquanto instruções simples e regulares facilitam a decodificação e o uso de *pipeline* (SILVA; BORGES, 2024).

Por isso, entender RISC e CISC ajuda a explicar o desempenho, o consumo de energia e as escolhas do compilador. Para quem desenvolve software, mostra também por que o mesmo código em C gera sequências de instruções diferentes em cada arquitetura.

1.3 Objetivos

1.3.1 Objetivo geral

Comparar as arquiteturas RISC e CISC, analisando suas características e aplicações, por meio de um estudo de caso e da análise do Assembly gerado a partir de um mesmo código em C.

1.3.2 Objetivos específicos

- Apresentar os conceitos de ISA, RISC e CISC, a densidade de código, o *pipeline* e a convergência entre as duas filosofias.

- Compilar o mesmo código em C para x86-64 e ARM64 no Compiler Explorer, com os níveis de otimização `-O0` e `-O2`, e comparar o Assembly gerado.
- Discutir os resultados da análise estática e suas limitações.
- Relacionar as diferenças entre as arquiteturas a cenários de uso em servidores e dispositivos móveis.

1.4 Organização do trabalho

O Capítulo 2 apresenta a fundamentação teórica. O Capítulo 3 traz os diagramas da arquitetura. O Capítulo 4 descreve a prova de conceito, com o código, o Assembly gerado e as métricas. O Capítulo 5 discute os resultados e suas limitações. O Capítulo 6 reúne as conclusões.

2 Fundamentação teórica

2.1 Arquitetura de conjunto de instruções (ISA)

A ISA (Instruction Set Architecture) é a interface que separa o hardware do software. Ela define o formato das instruções (opcode e operandos), os modos de endereçamento, os registradores visíveis ao programador e o modelo de memória. É a ISA que garante que um mesmo código compilado possa rodar em processadores de diferentes fabricantes, desde que compartilhem a mesma arquitetura (STALLINGS, 2017).

2.2 Filosofia RISC

A filosofia RISC (*Reduced Instruction Set Computer*) baseia-se na simplicidade. Seus princípios incluem: instruções de tamanho fixo, poucos formatos de instrução e a separação estrita entre operações aritméticas e acessos à memória (arquitetura *load/store*). Apenas as instruções de carga e armazenamento acessam a memória; o resto opera apenas sobre registradores. Isso facilita a implementação de *pipelines* profundos e decodificadores mais simples e rápidos (STALLINGS, 2017).

2.3 Filosofia CISC

2.3.1 Arquitetura memória-memória

Os processadores com arquitetura memória-memória possuem instruções aritméticas e lógicas onde todos os operandos são referenciados explicitamente. Ou seja, essas instruções possuem um total de três operandos, que podem estar localizados na memória. Apesar de, em grande parte dos casos, ainda ser necessário fazer uso de registradores, as instruções aritméticas e lógicas desses processadores têm a capacidade de referenciar todas as variáveis diretamente na memória (SILVA; BORGES, 2024).

Normalmente, o código gerado pelos compiladores para essas máquinas é bastante eficiente, com um número bem menor de instruções de máquina. Em contrapartida, resultam em arquiteturas com implementação bastante complexa e sem possibilidade do uso de técnicas de otimização da execução das instruções, como o *pipeline*. Como consequência, têm baixo desempenho quando comparadas com outras arquiteturas que adotaram soluções mais modernas, como os processadores RISC. Esse tipo de arquitetura é característico dos processadores CISC. Um exemplo de computador desse tipo é o VAX-11/780, um processador bastante comercializado nas décadas de 1970 e 1980 (SILVA; BORGES, 2024).

2.3.2 Arquitetura registrador-memória

Os processadores com arquitetura registrador-memória normalmente possuem instruções aritméticas e lógicas com dois operandos, sendo que um deles está em memória e o outro em registrador. O resultado da operação é escrito automaticamente no mesmo registrador. Os exemplos clássicos desse tipo de arquitetura são o IBM 360 e os processadores da linha Intel x86. Em sua maioria, também são consideradas arquiteturas do tipo CISC, sendo que os processadores da linha x86, modernamente, implementam um tradutor que, em tempo de execução, faz a tradução de instruções x86 convencionais para instruções de um processador RISC implementado internamente (SILVA; BORGES, 2024).

Código 2.1 – Exemplo de sequência em arquitetura registrador-memória (soma de A e B armazenada em C)

```
LOAD R1, A
ADD R1, B
STORE C, R1
```

2.3.3 Características da arquitetura CISC

Uma outra maneira de melhorar o desempenho é diminuir o número total de instruções (N_i) gastas para traduzir um programa da linguagem de alto nível para a linguagem de máquina. Assim, instruções cada vez mais complexas eram implementadas no conjunto de instruções do processador. Essas instruções podiam realizar de uma só vez várias operações complexas, acessando o maior número possível de operandos de uma única vez, com vários e diversos modos de endereçamento possíveis.

Essa aproximação foi adotada pelas arquiteturas CISC (*Complex Instruction Set Computer*), presentes nas primeiras arquiteturas de processadores, incluindo as dos primeiros microprocessadores, até o início da década de 1980. Exemplos de arquitetura CISC são os antigos processadores x86 da Intel e o VAX-11/780, entre outros computadores clássicos.

Resumindo, as arquiteturas CISC são caracterizadas por instruções complexas, que frequentemente demandam um grande número e variável de ciclos de relógio para serem executadas. Essas instruções possuem formatos variáveis e diversos modos de endereçamento dos operandos. Vários tipos de instruções podem acessar os seus operandos diretamente na memória. Como resultado, há uma dificuldade grande para o uso de técnicas como *pipeline* na implementação desses processadores. A taxa média de execução das instruções por ciclo tende a ser bastante superior a 1 ciclo por instrução (CPI) e a unidade de controle desses processadores é, em geral, microprogramada. Como grande vantagem, os compiladores podem gerar códigos compactos, com um reduzido número de instruções (SILVA; BORGES, 2024).

2.4 Densidade de código e complexidade de decodificação

Um dos objetivos da arquitetura de um processador é permitir a execução dos programas o mais rapidamente possível. Há várias técnicas que são empregadas para isso, mas, de uma forma simplificada, o tempo de execução de um programa pode ser definido pela seguinte equação:

$$T_p = C_i \times T_c \times N_i \tag{2.1}$$

Em que:

- T_p é o tempo de execução do programa;
- C_i são os ciclos gastos em média por instrução;
- T_c é o tempo de duração de um ciclo;
- N_i é o número de instruções do programa.

O C_i também é conhecido como CPI (Ciclos por Instrução) e define o número médio de ciclos gastos por cada instrução. É um valor que varia de acordo com cada programa, por exemplo, se possui mais instruções inteiras ou de ponto flutuante, além do tipo da arquitetura do processador. O CPI é obtido dividindo-se o número total de ciclos de relógio gastos para executar um programa pelo total de instruções executadas. O seu inverso é o IPC (Instruções por Ciclo), mais usado modernamente nas arquiteturas superescalares, para indicar o número de instruções, normalmente maior do que um, que podem ser executadas a cada ciclo de relógio do processador. O tempo de ciclo (T_c) é o inverso da frequência do relógio f, segundo a fórmula:

$$T_c = \frac{1}{f} \tag{2.2}$$

Por exemplo, se a frequência do relógio for 100 MHz (100×10^6 Hz), o tempo de ciclo será de 10 ns (10×10^{-9} s). Logo, quanto maior for a frequência do relógio, menor será o tempo de ciclo e menor será o tempo total de execução dos programas.

Em contrapartida, nas arquiteturas CISC o número de ciclos para executar cada instrução é muito maior, e um hardware mais complexo resultava em frequências de relógio menores, em virtude do longo tempo necessário para a decodificação das instruções (SILVA; BORGES, 2024).

2.5 Consumo de energia e contagem de transistores

Durante algum tempo os projetistas buscaram reduzir o tempo de execução dos programas aumentando ao máximo a frequência do relógio do processador, a partir de diversas técnicas de projeto, por exemplo, o *superpipelining*. Entretanto, essa abordagem

tem um limite em função do atraso mínimo dos componentes do processador, assim como da potência máxima que podia ser dissipada pelo conjunto encapsulamento e dissipador (SILVA; BORGES, 2024).

Historicamente, acreditava-se que processadores CISC, por serem mais complexos, consumiriam mais energia. No entanto, em processadores modernos, o consumo está mais atrelado à tensão de operação, à frequência e à eficiência do *pipeline* do que à classificação RISC ou CISC. Decodificadores complexos gastam mais energia, mas o uso extensivo de caches e a execução fora de ordem (*out-of-order*) também elevam o consumo em ambas as arquiteturas.

2.6 Pipeline

O *pipeline* é uma técnica de implementação de processadores que permite a sobreposição temporal das diversas fases de execução das instruções. Essa técnica aumenta o número de instruções executadas simultaneamente e a taxa de instruções iniciadas e terminadas por unidade de tempo. Contudo, o *pipeline* não reduz o tempo gasto para completar cada instrução individualmente, podendo até mesmo aumentá-lo em alguns casos (SILVA; BORGES, 2024).

2.7 Convergência moderna

Atualmente, os processadores da Intel são uma arquitetura híbrida, externamente compatíveis com o código da arquitetura x86, mas com as instruções traduzidas, em tempo de execução, para as instruções RISC da arquitetura interna, que efetivamente executa os programas (SILVA; BORGES, 2024, p. 74).

Arquiteturas RISC modernas, como o ARM, também adicionaram instruções complexas para processamento de mídia (como NEON e SVE), mostrando que a fronteira entre as filosofias se tornou tênue (ARM Limited, 2024).

3 Mapeamento e diagrama arquitetural

3.1 Fluxo de execução de instruções

O ciclo básico de execução de qualquer processador é composto por três etapas fundamentais: busca (*fetch*), decodificação (*decode*) e execução (*execute*). O contador de programa (PC) armazena o endereço da próxima instrução a ser executada. A unidade de controle busca essa instrução na memória e a decodifica para saber qual operação realizar. Em seguida, a ULA (Unidade Lógica e Aritmética) executa a operação, e o resultado pode ser armazenado em um registrador ou na memória.

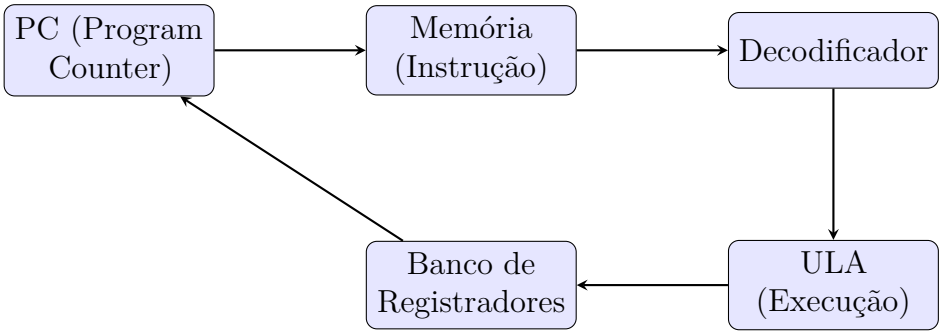


Figura 1 – Ciclo de busca, decodificação e execução

3.2 Fluxo de instrução em RISC e em CISC

Em arquiteturas RISC, a instrução lida da memória é decodificada diretamente pelo hardware, pois possui formato fixo e regular, sendo executada logo em seguida. Em arquiteturas CISC, especialmente no x86, a instrução complexa lida da memória passa por um tradutor que a converte em micro-operações menores (que se parecem com instruções RISC). São essas micro-operações que efetivamente entram no pipeline de execução.

3.3 Registradores das arquiteturas da PoC

Para entender o Assembly gerado na prova de conceito, é preciso conhecer os registradores de cada arquitetura:

- **x86-64:** Os registradores de uso geral têm 64 bits e incluem `rax`, `rbx`, `rcx`, `rdx`, `rsi`, `rdi`, `rbp` e `rsp` (este último o ponteiro de pilha). O `rax` é frequentemente usado para acumuladores e retorno de funções.
- **ARM64:** Possui 31 registradores de uso geral, nomeados de `x0` a `x30` (64 bits), além do `sp` (stack pointer) e `pc` (program counter). Quando o processador precisa operar com 32 bits, utiliza a visão `w0` a `w30` (ex: `w0` é a metade inferior de `x0`).

3.4 Diagrama de pipeline comparativo

O pipeline moderno é dividido em estágios (IF, ID, EX, MEM, WB). O RISC foi desenhado para preencher esses estágios com facilidade devido ao tamanho fixo das instruções, o que facilita a sobreposição. O CISC moderno também usa pipeline, mas necessita do estágio extra de tradução para micro-operações antes de entrar na execução.

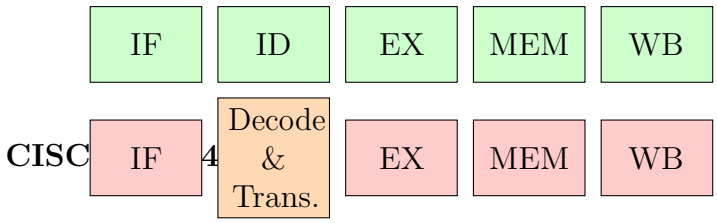


Figura 2 – Comparação de pipelines: RISC direto vs. CISC com tradução para micro-operações

4 Metodologia e desenvolvimento

4.1 Ambiente e ferramentas

Utilizou-se o Compiler Explorer (Godbolt) (Compiler Explorer, 2026) para compilar o mesmo código em C para duas arquiteturas. Os compiladores foram o *x86-64 gcc 16.1* e o *ARM64 GCC 16.1.0*, da mesma família e versão principal, para que a diferença observada venha da arquitetura e não do compilador. Foram usados os níveis de otimização `-O0` e `-O2`. O acesso à ferramenta ocorreu em 4 de novembro de 2026.

4.2 Arquiteturas selecionadas

Foram selecionadas a arquitetura x86-64, classificada como CISC, e a ARM64, classificada como RISC (STALLINGS, 2017, p. 23–28)(TANENBAUM; AUSTIN, 2013, p. 31–36). Em processadores modernos essa distinção é sutil, como discutido no Capítulo 2.

4.3 Código-fonte em C

O trecho analisado soma os elementos positivos de um vetor. Ele reúne laço de repetição, condição, leitura da memória e acumulação em uma variável.

```c
// PoC Tema 5: soma dos elementos positivos de um vetor.
int soma_positivos(int *v, int n) {
    int soma = 0;
    for (int i = 0; i < n; i++) {
        if (v[i] > 0)
            soma += v[i];
    }
    return soma;
}
```

Código 4.1 – Código em C da PoC

4.4 Assembly gerado

4.4.1 x86-64

```
"soma_positivos":
        push    rbp
        mov     rbp, rsp
        mov     QWORD PTR [rbp-24], rdi
        mov     DWORD PTR [rbp-28], esi
        mov     DWORD PTR [rbp-4], 0
```

```
        mov     DWORD PTR [rbp-8], 0
        jmp     .L2
.L4:
        mov     eax, DWORD PTR [rbp-8]
        cdqe
        lea     rdx, [0+rax*4]
        mov     rax, QWORD PTR [rbp-24]
        add     rax, rdx
        mov     eax, DWORD PTR [rax]
        test    eax, eax
        jle     .L3
        mov     eax, DWORD PTR [rbp-8]
        cdqe
        lea     rdx, [0+rax*4]
        mov     rax, QWORD PTR [rbp-24]
        add     rax, rdx
        mov     eax, DWORD PTR [rax]
        add     DWORD PTR [rbp-4], eax
.L3:
        add     DWORD PTR [rbp-8], 1
.L2:
        mov     eax, DWORD PTR [rbp-8]
        cmp     eax, DWORD PTR [rbp-28]
        jl      .L4
        mov     eax, DWORD PTR [rbp-4]
        pop     rbp
        ret
```

Código 4.2 – Assembly x86-64, gcc 16.1, -O0

```
"soma_positivos":
        test    esi, esi
        jle     .L5
        mov     esi, esi
        xor     edx, edx
        lea     rsi, [rdi+rsi*4]
.L4:
        mov     eax, DWORD PTR [rdi]
        xor     ecx, ecx
        test    eax, eax
        cmovs   eax, ecx
        add     rdi, 4
        add     edx, eax
        cmp     rdi, rsi
        jne     .L4
        mov     eax, edx
        ret
.L5:
```

```
19          xor     edx, edx
20          mov     eax, edx
21          ret
```

Código 4.3 – Assembly x86-64, gcc 16.1, -O2

4.4.2 ARM64

```
1  soma_positivos:
2          sub     sp, sp, #32
3          str     x0, [sp, 8]
4          str     w1, [sp, 4]
5          str     wzr, [sp, 28]
6          str     wzr, [sp, 24]
7          b       .L2
8  .L4:
9          ldrsw   x0, [sp, 24]
10         lsl     x0, x0, 2
11         ldr     x1, [sp, 8]
12         add     x0, x1, x0
13         ldr     w0, [x0]
14         cmp     w0, 0
15         ble     .L3
16         ldrsw   x0, [sp, 24]
17         lsl     x0, x0, 2
18         ldr     x1, [sp, 8]
19         add     x0, x1, x0
20         ldr     w0, [x0]
21         ldr     w1, [sp, 28]
22         add     w0, w1, w0
23         str     w0, [sp, 28]
24 .L3:
25         ldr     w0, [sp, 24]
26         add     w0, w0, 1
27         str     w0, [sp, 24]
28 .L2:
29         ldr     w1, [sp, 24]
30         ldr     w0, [sp, 4]
31         cmp     w1, w0
32         blt     .L4
33         ldr     w0, [sp, 28]
34         add     sp, sp, 32
35         ret
```

Código 4.4 – Assembly ARM64, gcc 16.1.0, -O0

```
1  soma_positivos:
```

```
2           cmp     w1, 0
3           ble     .L5
4           mov     x3, x0
5           add     x1, x0, w1, uxtw 2
6           mov     w0, 0
7   .L4:
8           ldr     w2, [x3], 4
9           bic     w2, w2, w2, asr #31
10          add     w0, w0, w2
11          cmp     x3, x1
12          bne     .L4
13          ret
14  .L5:
15          mov     w0, 0
16          ret
```

Código 4.5 – Assembly ARM64, gcc 16.1.0, -O2

4.4.3 Comentários sobre as listagens

x86-64, `-O0`. Todas as variáveis ficam na pilha, acessadas por endereços relativos a `rbp`. O prólogo `push rbp` e `mov rbp, rsp` monta o quadro de pilha. O teste do `for` vira `cmp` seguido de `jl .L4`, e o `if` vira `test` seguido de `jle .L3`, que salta sobre a soma quando o elemento não é positivo. A acumulação é feita por `add DWORD PTR [rbp-4], eax`, uma única instrução que lê e escreve a memória.

ARM64, `-O0`. O quadro de pilha é reservado por `sub sp, sp, #32`. Apenas `ldr`, `ldrsw` e `str` acessam a memória; `add`, `lsl` e `cmp` operam sobre registradores. O endereço de `v[i]` é calculado por `lsl` e `add`. O `for` vira `cmp` e `blt .L4`, e o `if` vira `cmp` e `ble .L3`. A mesma acumulação exige três instruções: `ldr`, `add` e `str`.

x86-64, `-O2`. As variáveis passam para registradores e a pilha deixa de ser usada. O `if` não gera salto: `test eax, eax` e `cmovs eax, ecx` substituem o valor por zero quando ele é negativo. O laço percorre o vetor por ponteiro, com `add rdi, 4` e `cmp rdi, rsi` seguido de `jne .L4`. O único acesso à memória é `mov eax, DWORD PTR [rdi]`.

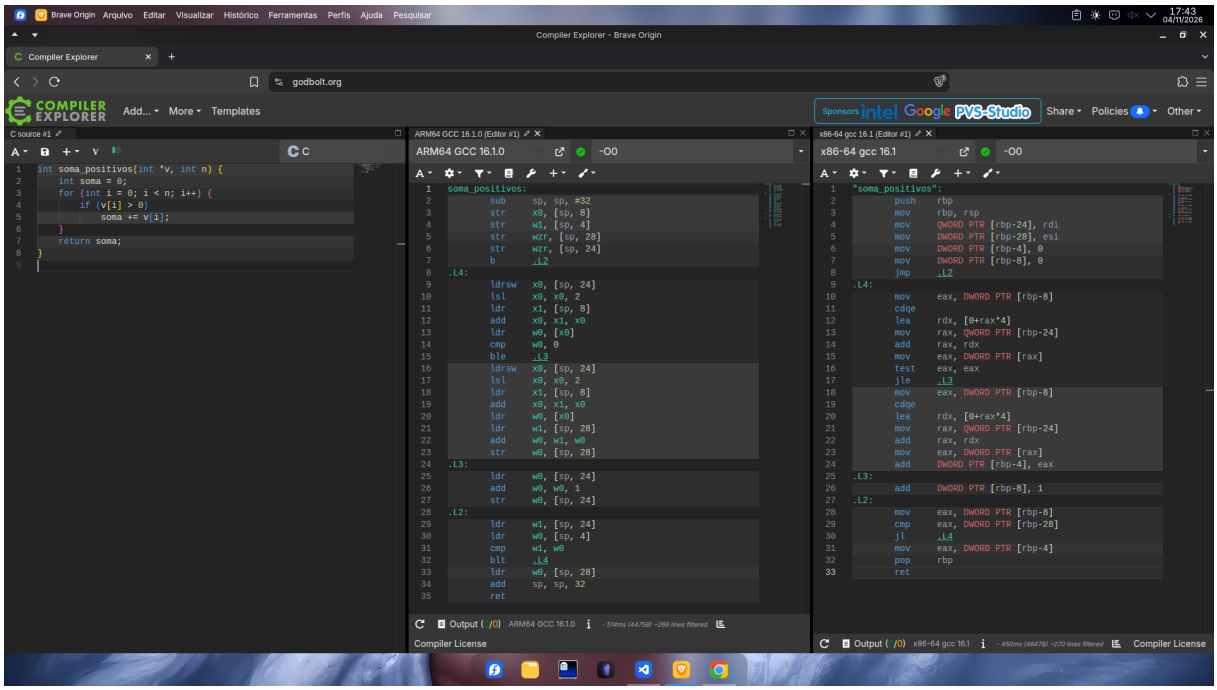
ARM64, `-O2`. A instrução `ldr w2, [x3], 4` carrega o elemento e avança o ponteiro em uma só operação (pós-incremento). O `if` também some como salto: `bic w2, w2, w2, asr #31` zera os valores negativos. O laço fecha com `cmp x3, x1` e `bne .L4`.

4.5 Passo a passo da PoC

1. Abrir o Compiler Explorer e escolher a linguagem C.
2. Colar o código do Código 4.1 no editor.
3. Selecionar o compilador x86-64 gcc 16.1 e informar a opção `-O0`.

4. Adicionar um segundo painel de compilação (*Add new*) com o ARM64 GCC 16.1.0 e a mesma opção `-O0`.
5. Copiar o Assembly de cada painel e salvar em arquivo.
6. Repetir os passos 3 a 5 com a opção `-O2`.
7. Registrar a tela de cada configuração (Figuras 3 e 4).

Figura 3 – Compiler Explorer com x86-64 e ARM64, opção -O0



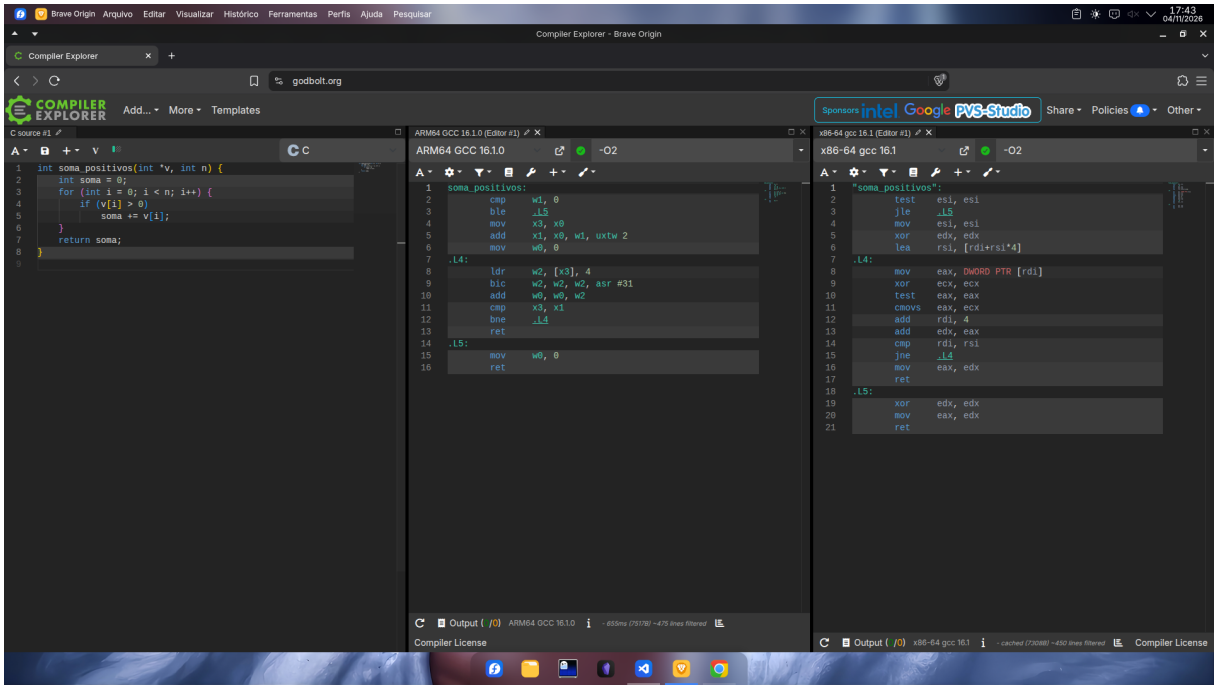
Fonte: elaborado pelos autores no Compiler Explorer.

4.6 Métricas coletadas

Foram coletadas métricas estáticas, contadas sobre o texto do Assembly gerado:

- **Instruções:** linhas de instrução, sem contar rótulos e o nome da função. O retorno é contado.
- **Saltos:** instruções de desvio condicional e incondicional. O retorno não é contado.
- **Acessos à memória:** em ARM64, as instruções de carga e armazenamento. Em x86-64, as instruções com operando de memória, exceto `lea`, que apenas calcula o endereço.
- **Registradores utilizados:** registradores de uso geral distintos, tratando as visões de 32 e 64 bits como um só registrador (por exemplo, `eax` e `rax`). Ponteiro de pilha

Figura 4 – Compiler Explorer com x86-64 e ARM64, opção -O2



Fonte: elaborado pelos autores no Compiler Explorer.

(`sp`, `rsp`) e registrador zero (`wzr`) não são contados; o `rbp`, usado como base do quadro de pilha, é contado.

- **Bytes de código:** tamanho da função lido na saída binária do Compiler Explorer. Em ARM64, cada instrução ocupa 4 bytes; em x86-64, o tamanho varia. Em `-O2`, os bytes incluem 3 instruções `nop` de alinhamento em cada arquitetura, que não entram na contagem de instruções.

Os valores estão na Tabela 1, no Capítulo 5.

4.7 Limitação do método

O Godbolt realiza análise estática. Ele mostra as instruções geradas, mas não executa o programa e não mede ciclos reais. Contagem de instruções não equivale a desempenho. Os resultados valem para este trecho de código e para as versões de compilador usadas.

5 Discussão dos resultados

5.1 Métricas obtidas

Tabela 1 – Comparação estática do código gerado (gcc 16.1)

	`-O0`		`-O2`	
Métrica	x86-64	ARM64	x86-64	ARM64
Instruções	29	31	18	13
Saltos	3	3	2	2
Acessos à memória	15	17	1	1
Registradores utilizados	5	2	5	4
Bytes de código	95	124	66	64

Fonte: elaborado pelos autores com dados do Compiler Explorer. Em `-O2`, os bytes incluem 3 instruções `nop` de alinhamento em cada arquitetura, não contadas em "Instruções".

5.2 Análise da comparação

Em `-O0`, o ARM64 executa 31 instruções contra 29 do x86-64, e ocupa mais bytes (124 contra 95). A largura fixa de 4 bytes do ARM64 contrasta com as instruções de tamanho variável do x86-64. Em `-O2`, o ARM64 usa 13 instruções contra 18, e os bytes quase empatam (64 contra 66). Nos dois níveis, a mudança de otimização altera mais a contagem do que a mudança de arquitetura.

5.3 Do código em C ao hardware

O laço `for` vira uma comparação seguida de salto condicional: `cmp` e `jl` no x86-64, `cmp` e `blt` no ARM64. Em `-O0`, a acumulação `soma += v[i]` mostra a diferença entre as filosofias. No x86-64, `add DWORD PTR [rbp-4], eax` lê e escreve a memória em uma instrução. No ARM64, são necessárias `ldr`, `add` e `str`, pois apenas carga e armazenamento acessam a memória. Em `-O2`, as variáveis passam para registradores e o `if` deixa de gerar salto: `cmovs` no x86-64 e `bic` no ARM64.

5.4 Limitações do método

O Godbolt fornece análise estática. Ele mostra as instruções geradas, mas não executa o programa e não mede ciclos reais, tempo de CPU, cache, previsão de saltos nem execução fora de ordem. Contagem de instruções e de bytes não equivale a desempenho. Os resultados valem para este trecho de código e para o gcc 16.1.

5.5 Limitações físicas

O consumo de energia e a dissipação de calor limitam o aumento da frequência de operação dos processadores. Por isso, o ganho de desempenho passou a depender de paralelismo e de eficiência por instrução (PATTERSON; HENNESSY, 2017, p. 24–28). A contagem de transistores também tem custo de área e de energia, e decodificadores de instruções irregulares ocupam mais recursos que decodificadores de instruções regulares (STALLINGS, 2017, p. 408–410, 560–562).

Atribuir diferenças de consumo apenas à classificação RISC ou CISC é uma simplificação, pois cache, pipeline e processo de fabricação também influenciam o resultado. Na PoC, o ARM64 usa instruções de largura fixa de 4 bytes, enquanto o x86-64 usa instruções de tamanho variável. Esse contraste de formato é o que torna a decodificação do x86-64 mais complexa (TANENBAUM; AUSTIN, 2013, p. 289–292). A PoC, porém, é estática: não mede consumo, área nem frequência.

5.6 Estudo de caso: servidores e dispositivos móveis

Para o mesmo trecho em C, a PoC mostra o que cada ISA exige do compilador e do hardware. O x86-64 permite operar diretamente sobre a memória, como em `add DWORD PTR [rbp-4], eax`, e tem instruções de tamanho variável (Intel Corporation, 2024). O ARM64 separa o acesso à memória da aritmética (`ldr`, `add`, `str`) e tem instruções de tamanho fixo (ARM Limited, 2024). Esses traços orientam o uso de cada arquitetura, mas não bastam para afirmar qual é melhor em servidores ou em dispositivos móveis, pois isso depende de energia, desempenho medido e compatibilidade de software.

No mercado, as duas arquiteturas coexistem. Em servidores x86-64, a Intel e a AMD dividem a receita de CPUs para servidores: no primeiro trimestre de 2026, a Intel ficou com 53,8% e a AMD com 46,2% (VITORIO, 2026). Em nuvem, a Amazon projetou processadores ARM próprios, os Graviton, oferecidos como máquinas virtuais na AWS (WILLIAMS, 2018). Em computadores pessoais, a Apple anunciou em 2020 a troca dos processadores Intel por chips próprios de arquitetura ARM, a mesma usada em iPhones e iPads (VIEIRA, 2020).

5.7 Impacto no desenvolvimento de software de alto desempenho

A ISA aparece para o programador por meio do compilador. O mesmo código C gerou sequências diferentes em x86-64 e ARM64, e o programa precisa ser recompilado para cada arquitetura (Compiler Explorer, 2026). Na PoC, a troca de `-O0` para `-O2` alterou a contagem de instruções mais do que a troca de arquitetura (Tabela 1). Para quem escreve software de alto desempenho, isso significa que as opções do compilador e a escolha de algoritmos pesam tanto quanto a ISA.

Os efeitos de cache, previsão de saltos e execução fora de ordem não foram medidos neste trabalho. A análise de um programa real exige medição de tempo e de contadores de hardware, além do Assembly. Detalhes de cada instrução devem ser conferidos nos manuais das arquiteturas (Intel Corporation, 2024; ARM Limited, 2024).

6 Conclusão

6.1 Síntese do aprendizado

Este trabalho comparou as arquiteturas RISC e CISC, utilizando a análise estática do Assembly gerado pelo Compiler Explorer como prova de conceito. Os objetivos foram atingidos ao demonstrar que, embora o x86-64 (CISC) tenha gerado menos instruções em nível de otimização zero (-O0), ele utiliza instruções de tamanho variável que tornam a decodificação mais complexa. Por outro lado, o ARM64 (RISC) gerou mais instruções, porém com largura fixa e separação clara entre operações de memória (load/store) e de registradores.

6.2 Lições aprendidas

A principal lição é que a divisão entre RISC e CISC não é mais uma fronteira rígida. Os processadores modernos convergiram: o x86-64 traduz instruções complexas em micro-operações internas (tornando-se RISC por dentro), enquanto o ARM64 adiciona extensões complexas (como vetorização) para ganhar desempenho. Além disso, a PoC mostrou que o nível de otimização do compilador (de -O0 para -O2) altera drasticamente o código gerado, reduzindo a contagem de instruções e muitas vezes superando o impacto da própria mudança de arquitetura. O uso de instruções como `cmovs` e `bic` elimina saltos condicionais, otimizando o pipeline.

6.3 Implicações práticas

Para o desenvolvimento de software de alto desempenho, entender a arquitetura é fundamental para escolher a melhor linguagem, as flags de compilação corretas e as estruturas de dados adequadas. O programador precisa saber que o mesmo código em C terá um comportamento de hardware diferente em um servidor x86 e em um dispositivo móvel ARM, e que o compilador fará otimizações agressivas quando solicitado. Compreender como o laço `for` se transforma em saltos e como a memória é acessada permite escrever códigos que exploram melhor a localidade e o poder de processamento dos processadores modernos.

Referências

ARM Limited. *Arm Architecture Reference Manual for A-profile architecture*. [S.l.], 2024. Disponível em: https://developer.arm.com/documentation/ddi0487/latest/. Acesso em: 6 nov. 2026.

Compiler Explorer. *Compiler Explorer*. 2026. Ferramenta online. Disponível em: https://godbolt.org. Acesso em: 4 nov. 2026.

Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*. [S.l.], 2024. Disponível em: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html. Acesso em: 6 nov. 2026.

MONTEIRO, M. A. *Introdução à Organização de Computadores*. 5. ed. Rio de Janeiro: LTC, 2007.

PATTERSON, D. A.; HENNESSY, J. L. *Organização e Projeto de Computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: Elsevier, 2017.

RISC-V International. *The RISC-V Instruction Set Manual*. [S.l.], 2019. Disponível em: https://riscv.org/technical/specifications/. Acesso em: 6 nov. 2026.

SILVA, G. P. d.; BORGES, J. A. d. S. *Arquitetura e Organização de Computadores: uma introdução*. Rio de Janeiro: LTC, 2024. E-book. ISBN 9788521638667. Disponível em: https://integrada.minhabiblioteca.com.br/reader/books/9788521638667/. Acesso em: 4 nov. 2026.

STALLINGS, W. *Arquitetura e Organização de Computadores*. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. *Organização Estruturada de Computadores*. 6. ed. São Paulo: Pearson Universidades, 2013.

VIEIRA, N. *WWDC 2020: Apple anuncia troca da Intel por processadores próprios*. 2020. Disponível em: https://canaltech.com.br/wwdc/wwdc-2020-apple-anuncia-troca-da-intel-por-processadores-proprios-166845/. Acesso em: 4 nov. 2026.

VITORIO, T. *Nvidia projeta US$ 20 bilhões em CPUs em 2026, 19% a mais do que a Intel faturou em 2025*. 2026. Disponível em: https://exame.com/inteligencia-artificial/nvidia-projeta-us-20-bilhoes-em-cpus-em-2026-19-a-mais-do-que-a-intel-faturou-em-2025/. Acesso em: 4 nov. 2026.

WEBER, R. F. *Fundamentos de Arquitetura de Computadores*. 4. ed. Porto Alegre: Bookman, 2012.

WILLIAMS, C. *A rumble in Amazon's jungle: AWS now rents out homegrown 64-bit Arm server processors*. 2018. Disponível em: https://www.theregister.com/off-prem/2018/11/27/a-rumble-in-amazons-jungle-aws-now-rents-out-homegrown-64-bit-arm-server-processors/832831. Acesso em: 4 nov. 2026.